

便携式灯条基站API

一、前言

本手册API内容是灯条基站SDK文件和便携式基站通讯的应用程序接口函数定义，旨在使集成便携式灯条基站相关控制功能变得更加方便和高效。
用户可以使用开发工具如： Eclipse，AndroidStudio等开发工具进行应用开发。
该库采用 Java 语言编写并封装成标准的 jar 包。

二、开发说明

2.1 根据本开发资料中（集成开发需要用到文件）目录完成开发环境搭建。

2.2 集成开发Jar放置说明:

[OTG连接]

将“集成开发需要用到文件”目录下所有jar文件(mina-core-2.0.20.jar、slf4j-api-1.7.26.jar、UHFReader.jar、CH34xUARTDriver.jar、d2xx.jar)放于Android项目libs目录下，清理刷新项目。

[原生串口连接]

将“集成开发需要用到文件”目录下所有jar文件(mina-core-2.0.20.jar、slf4j-api-1.7.26.jar、UHFReader.jar)放于Android项目libs目录下，拷贝armeabi文件夹到libs目录下，之后刷新代码，清理刷新项目。

[蓝牙连接]

将“集成开发需要用到文件”目录下所有jar文件(mina-core-2.0.20.jar、slf4j-api-1.7.26.jar、UHFReader.jar、feasyblue.jar)放于Android项目libs目录下，拷贝armeabi文件夹、arm64-v8a文件夹、armeabi-v7a文件夹到libs目录下，之后刷新代码，清理刷新项目。

2.3 定义基站库操作接口类对象且实例化

- 实例化一次全局调用

```
public static Reader Readercontroller = new Reader(this);
```

2.4 实现 MultiLableCallBack 接口

```
public class SrDemo implements MultiLableCallBack
```

实现回调方法method(基站所有响应数据都会通过该方法返回)

```
@Override
    public void method(String data)//data以","划分为数组，为基站响应数据
```

实现回调方法ReaderNotice(暂无需处理)

```
@Override
    public void ReaderNotice(String NoticeMsg)
```

2.5 项目配置说明

注：应用层APP需添加以下权限(OTG连接)

```
<uses-permission android:name="android.permission.USB" />
<uses-permission android:name="android.permission.USB_PERMISSION" />
```

注：应用层APP需添加以下权限(蓝牙连接)

```
<uses-permission android:name="android.permission.BLUETOOTH" android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />
<!-- Android 12 需要以下权限 -->
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
<uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
```

蓝牙连接build.gradle配置

```
dependencies {
    // 需要导入Kotlin 协程环境
    def coroutines_version = "1.4.2"
    implementation "org.jetbrains.kotlinx:kotlinx-coroutines-core:$coroutines_version"
}
```

```
defaultConfig {
    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }
    ndk{
        abiFilters "armeabi","armeabi-v7a","x86","arm64-v8a"
    }
}

splits{
    abi {
        include "armeabi","armeabi-v7a","x86","arm64-v8a"
    }
}
```

三、连接说明

3.1 注册USB广播接收器(OTG连接)

```
public UsbPermission usbPermission;

/**
 * 注册USB广播接收器
 */
public void registerUsbReceiver()
{
    usbPermission = new UsbPermission(context);//USB 权限相关辅助类
    usbPermission.setReaderController(ReaderController);
    usbPermission.detectUsbWithBroadcast();

    usbPermission.checkUsbDevicePermission();
}
```

- 注：UsbPermission类代码请参考提供源码文件

3.2 打开Android OTG扩展连接(CH34x)

方法

```
public Boolean OpenOtgPort_CH341(Context context)
```

参数说明

参数名	必选	类型	说明
context	是	Context	当前程序上下文对象；this



返回值说明

- true:连接成功；false:连接失败

备注

- 如连接失败，可查看Reader类中String变量OpenOtgSerialPortErrorMsg来得到错误信息

3.3 关闭Android OTG扩展连接(CH34x)

方法

```
public Boolean CloseOtgPort_CH341()
```

返回值说明

- true:关闭成功；false:关闭失败

3.4 打开Android OTG扩展连接(Cdc)

方法

```
public Boolean OpenOtgPort_Cdc(Context context)
```

参数说明

参数名	必选	类型	说明
context	是	Context	当前程序上下文对象；this

返回值说明

- true:连接成功；false:连接失败

3.5 关闭Android OTG扩展连接(Cdc)

方法

```
public Boolean CloseOtgPort_Cdc()
```

返回值说明

- true:关闭成功；false:关闭失败

3.6 打开Android原生串口连接

方法

```
public Boolean OpenSerialPort_Android(String SerialName)
```

参数说明

参数名	必选	类型	说明
SerialName	是	String	串口号; 例 “/dev/ttyS3”

返回值说明

- true:连接成功；false:连接失败

备注

- 如连接失败，可查看Reader类中String变量OpenAndroidSerialPortErrorMsg来得到错误信息

3.7 关闭Android原生串口连接

方法

```
public Boolean CloseSerialPort_Android()
```

返回值说明

- true:关闭成功；false:关闭失败

3.8 初始化蓝牙

方法

```
public void initBleHandle(Context context)
```

参数说明

参数名	必选	类型	说明
context	是	Context	上下文对象

-
- 注：蓝牙连接前请检查GPS、蓝牙是否已开启以及程序是否已取得GPS位置信息动态权限。可参考源码MainActivity下checkGpsEnabled()、checkBlePermissions()、checkBleEnabled()、checkGpsPermission()函数接口。

3.9 连接蓝牙基站

方法

```
public void connectBle(String bleName,boolean reconnectFlag,int filterRssi)
```

参数说明

参数名	必选	类型	说明
bleName	是	String	蓝牙名称；暂固定为“XY-FlashFind”即可
reconnectFlag	是	boolean	重连标志；固定为true即可

参数名	必选	类型	说明
filterRssi	是	int	蓝牙连接过滤信号值(值范围：-90~-30)；该值越接近0，则对基站信号要求越强。默认传-90即可

3.10 断开蓝牙基站

方法

```
public void disconnectBle()
```

3.11 更新蓝牙基站连接过滤信号值

方法

```
public void setBleFilterRssi(int filterRssi)
```

参数说明

参数名	必选	类型	说明
filterRssi	是	int	蓝牙连接过滤信号值(值范围：-90~-30)；该值越接近0，则对基站信号要求越强。默认传-90即可

3.12 获取蓝牙基站状态信息

方法

```
public BleStateMessage getBleStateMessage()
```

BleStateMessage说明

参数名	类型	说明
bleStateMessage.bleState	int	蓝牙基站状态 扫描中：BleStateTypes.Scanning 连接中：BleStateTypes.Connecting 已连接：BleStateTypes.Connected 已断开：BleStateTypes.DisConnected
bleStateMessage.bleRssi	int	蓝牙基站信号值 该值越接近0，则基站信号越强。 基站状态为已连接时该值有效

3.13 获取基站通讯对象

方法

```
public List<AsyncSocketState> GetClientInfo()
```

返回值说明

- 返回基站通讯对象集合，固定获取集合第一个元素即可。

四、接口说明

4.1 控制灯条(点亮灯条/停止灯条)

方法

```
public String SetAcoustOpticTagsWork(AsyncSocketState state,byte rgb,byte beepState,
List<byte[]> lightCodeList,byte workTime,byte intervalTime)
```

参数说明

参数名	说明
state	基站设备对象
rgb	灯条灯光控制值 最高bit控制是否闪灯(0:闪灯；1：长亮)

参数名	说明
	0x01:蓝色闪灯；0x81:蓝色长亮 0x02:绿色闪灯；0x82:绿色长亮 0x03:青色闪灯；0x83:青色长亮 0x04:红色闪灯；0x84:红色长亮 0x05:紫色闪灯；0x85:紫色长亮 0x06:黄色闪灯；0x86:黄色长亮 0x07:白色闪灯；0x87:白色长亮 0x08:不亮灯
beepState	声音控制； 0x01:发声；0x00:不发声
lightCodeList	灯条暗码ID号集合(4字节byte[])； 集合只包含一个byte[]{0x00,0x00,0x00,0x00}时即为控制全部灯条
workTime	灯条亮灯时长(单位：3秒)
intervalTime	灯条闪光间隔（单位：100毫秒）

返回值说明

- “0”：发送成功,“-4”：发送失败

4.2 控制灯条(多色点亮/多色停止)

方法

```
public String SetAcoustOpticTagsWork(AsyncSocketState state,List<byte[]> lightCodeList,byte beepState,
List<Map.Entry<Byte, Byte>> colorStates)
```

参数说明

参数名	说明
state	基站设备对象
lightCodeList	灯条暗码ID号集合(4字节byte[])； 集合中灯条ID数目不超过5
beepState	声音控制； 0x01:发声；0x00:不发声
colorStates	灯光控制； 集合元素Key值为控制亮灯颜色(1-7颜色对应，蓝绿青红紫黄白，8为熄灭该灯条所有颜色)

参数名	说明
	集合元素Value值为控制单个颜色亮起或熄灭(0:熄灭该颜色；1:亮起该颜色)

返回值说明

- “0”：发送成功,“-4”：发送失败

4.3 盘点灯条

方法

```
public String MultiAcoustOpticTags(AsyncSocketState state,int inventoryInterval,int inventoryCount,int inventoryTotal)
```

参数说明

参数名	说明
state	基站设备操作对象
inventoryInterval	盘点间隔时长(单位：1毫秒)
inventoryCount	盘点内置次数(建议默认值为3)
inventoryTotal	盘点总次数(建议默认值为20，最大值不超过100)



返回值说明

- “0”：发送成功,“-4”：发送失败

备注

- 盘点时间计算(值单位：秒)

```
int multiTimeForSecond = (1500 + inventoryInterval) * inventoryTotal + 10000 + 10000;
multiTimeForSecond = (multiTimeForSecond + (multiTimeForSecond / 60000) * 10000) / 1000;
```

4.4 更新分组

方法

```
public String UpdateAcoustOpticTagsGroup(AsyncSocketState state,byte[] tagID,byte label1,byte[] label2,byte[] label3)
```

参数说明

参数名	说明
state	基站设备操作对象
tagID	分组灯条号，16进制4字节灯条暗码
label1	分组标签1(长度1个字节) 快递公司信息；默认值FF
label2	分组标签2(长度2个字节) 货架号信息；默认值FFFF
label3	分组标签3(长度4个字节) 存放时间戳；默认值FFFFFFFF



返回值说明

- “0”：发送成功,“-4”：发送失败

4.5 点亮分组

方法

```
public String SetGroupAcoustOpticTagsWork(AsyncSocketState state,byte label1,byte[] label2,byte[] label3,
byte label1Rule,byte label2Rule,byte label3Rule,byte rgb,byte beepState,byte workTime,byte intervalTime)
```

参数说明

参数名	说明
state	基站设备操作对象
label1	分组标签1(长度1个字节) 快递公司信息；默认值FF
label2	分组标签2(长度2个字节) 货架号信息；默认值FFFF

参数名	说明
label3	分组标签3(长度4个字节) 存放时间戳；默认值FFFFFFFF
label1Rule	分组标签1匹配规则 0：= 1：> 2：< 3：≠ 4：∅(不参与匹配)
label2Rule	分组标签2匹配规则 值：同分组标签1匹配规则
label3Rule	分组标签3匹配规则 值：同分组标签1匹配规则
rgb	灯条灯光控制值
beepState	灯条是否发声控制值 0x01：需要发声；0x00：不需要发声
workTime	灯条亮灯时长(单位：3秒)
intervalTime	灯条闪光间隔（单位：100毫秒）

返回值说明

- “0”：发送成功,“-4”：发送失败

4.6 设置灯条心跳间隔

方法

```
public String SetAcoustOpticTagsHeart(AsyncSocketState state,int heartInterval)
```

参数说明

参数名	说明
state	基站设备操作对象
heartInterval	灯条心跳间隔(单位：2秒)

返回值说明

- “0”：发送成功,“-4”：发送失败



五、基站响应数据解析说明

5.1 基站响应数据解析示例

```
/**
 *接收基站设备响应数据
 */
@Override
public void method(String data)
{
    String[] result = data.split("\\,");
    byte type = toBytes(result[1])[0];
    switch (type)
    {
        case Types.ACOUSTO_OPTIC_TAGS_WORK://点亮灯条、盘点操作响应(收到该数据表示基站已接受该命令执行)
        case Types.ACOUSTO_OPTIC_TAGS_FUNCTION://更新灯条分组、点亮分组灯条响应(收到该数据表示基站已接受该命令执行)
            //处理数据
            break;
        case Types.ACOUSTO_OPTIC_TAGS_ALARM_RESPOND://灯条信息(常规)
            //处理数据
            break;
        =case Types.ACOUSTO_OPTIC_EXPANSION_TAGS_RESPOND://灯条主动上报信息(扩展)
            if (result[2].equals("1"))
            {
                switch(result[3])
                {
                    case "0"://开关量灯条信息
                        //处理数据
                        break;
                }
            }
            break;
        case Types.MODULE_LOG://模块日志
            //处理数据
            break;
    }
}

/**
 * 将16进制字符串转换为byte[]
 *
 * @param str
 * @return
 */
public static byte[] toBytes(String str) {
    if(str == null || str.trim().equals("")) {
        return new byte[0];
    }
    str = str.replace(" ", "");
```

```
byte[] bytes = new byte[str.length() / 2];
for(int i = 0; i < str.length() / 2; i++) {
    String subStr = str.substring(i * 2, i * 2 + 2);
    bytes[i] = (byte) Integer.parseInt(subStr, 16);
}
return bytes;
}
```

5.2 控制灯条/盘点灯条 操作响应

值	说明
string[0]	保留
string[1]	Cmd命令类型；此处为“F4”
string[2]	结果码 “1”:控制成功 其他值:控制失败



备注

- 如与基站连接正常，则在调用“控制灯条”或“盘点灯条”接口后，会收到该数据

5.3 更新灯条分组/点亮分组灯条 操作响应

值	说明
string[0]	保留
string[1]	Cmd命令类型；此处为“F9”
string[2]	结果码 “1”:控制成功 其他值:控制失败



备注

- 如与基站连接正常，则在调用“更新灯条分组”或“点亮分组灯条”接口后，会收到该数据

5.4 灯条信息响应(常规)

值	说明
string[0]	保留
string[1]	Cmd命令类型；此处为“F5”
string[2]	结果码 “1”:成功(表该数据有效) 其他值:失败
string[3]	灯条ID号
string[4]	灯条电压值(单位：mV)
string[5]	灯条版本
string[6]	灯条信号强度(越接近0越强)
string[7]	灯条动作； 是否发声：0不发声；1发声
string[8]	灯条动作； 是否闪灯：0闪灯；1长亮
string[9]	灯条动作； 是否多色：0单色；1多色
string[10]	灯条按键状态； 0无按键；1空闲时按键；2动作时按键
string[11]	灯条动作； 颜色值：低位到高位即从右往左为： 1蓝2绿3青4红5紫6黄7白8不亮灯，如含对应颜色，则对应bit为1



备注

- string[7]、string[8]、string[9]、string[10]都为“0”且string[11]也为“00000000”时，表示无动作。
- 如与基站连接正常，则在调用“盘点灯条”接口或人为按下灯条物理按键后，会收到该数据

5.5 灯条主动上报信息(扩展：开关量灯条)

值	说明
string[0]	保留

值	说明
string[1]	Cmd命令类型；此处为“F6”
string[2]	结果码 “1”:成功(表该数据有效) 其他值:失败
string[3]	SubCmd子命令类型；0表示为开关量灯条信息
string[4]	灯条ID号
string[5]	灯条电压值(单位：mV)
string[6]	灯条硬件版本
string[7]	灯条固件版本
string[8]	1号开关量状态
string[9]	2号开关量状态
string[10]	3号开关量状态
string[11]	4号开关量状态
string[12]	5号开关量状态
string[13]	6号开关量状态
string[14]	7号开关量状态
string[15]	8号开关量状态